

Tech Stack Fundamentals & Coding Mastery

The Exact Concepts, Live Code Snippets & SQL Queries They Will Ask You In The Interview

JavaScript ES6+

React 18 Hooks

Vite Tooling

Node.js Engine

Express.js APIs

SQL Queries & CRUD

⚡ Module 1: JavaScript Fundamentals (The Core Language)

They will test if you actually understand JavaScript before asking framework questions.

1. var vs let vs const (Scope & Hoisting)

Fundamental

WHAT IS IT? Three ways to declare variables in JavaScript.

WHY CARE? `var` is old (pre-2015/ES6) and dangerous because it ignores curly brace `{}` block scope and gets hoisted to the top as `undefined`.

THE RULE:

- **const** : Default choice! For values that never get reassigned. Block-scoped.
- **let** : Use only when the value *must* be reassigned (like loop counters). Block-scoped.
- **var** : Never use in modern code! Function-scoped.

```
const maxLoans = 7;           // Cannot be reassigned (maxLoans = 8 throws Error!)
let copies = 5;               // Can be changed: copies = copies - 1

if (true) {
  let blockScoped = "inside";
}
// console.log(blockScoped); // ReferenceError: blockScoped is not defined!
```

📄 INTERVIEW QUESTION: "WHAT IS THE DIFFERENCE BETWEEN == AND ===?"

"`=`" (loose equality) performs type coercion, meaning `'5' = 5` evaluates to `true`. `===` (strict equality) checks both value AND type without coercion, so `'5' === 5` is `false`. In modern code, we always use `===`."

2. Essential Array Methods (`.map`, `.filter`, `.find`, `.reduce`)

Must Know

In React, we never use old `for` loops to render lists. We use array methods:

```
const books = [
  { id: 'BK1', title: 'Clean Code', copies: 3, category: 'CS' },
  { id: 'BK2', title: 'Atomic Habits', copies: 0, category: 'Self-Help' }
];

// 1. .map() - Transforms each item into something new (returns new array of same length)
const titles = books.map(b => b.title); // ['Clean Code', 'Atomic Habits']

// 2. .filter() - Keeps only items that pass the true/false test
const available = books.filter(b => b.copies > 0); // Only Clean Code

// 3. .find() - Returns the FIRST matching item (or undefined)
const found = books.find(b => b.id === 'BK1'); // { id: 'BK1', title: 'Clean Code', ... }

// 4. .some() - Returns true if AT LEAST ONE item matches
const hasOutOfStock = books.some(b => b.copies === 0); // true
```

3. Asynchronous JavaScript: Promises & async / await

Core Concept

WHAT IS IT? JavaScript is single-threaded (executes one line at a time). When waiting for slow tasks (like fetching books from an API or reading files), JS runs them in the background so the browser doesn't freeze!

HOW ASYNC/AWAIT WORKS: `async/await` is syntactic sugar over Promises that lets you write asynchronous code that reads like normal synchronous top-to-bottom code.

```
// Using modern async/await with try/catch:
async function fetchLibraryBooks() {
  try {
    const response = await fetch('https://api.library.com/books');
    const data = await response.json();
    console.log('Books received:', data);
  } catch (error) {
    console.error('Failed to fetch books:', error);
  }
}
```



INTERVIEW QUESTION: "WHAT IS THE EVENT LOOP IN JAVASCRIPT?"

"JavaScript has a single Call Stack where code executes. When an asynchronous operation (like `fetch` or `setTimeout`) finishes, its callback goes into a Task Queue. The

EVENT LOOP

continuously checks: when the Call Stack is empty, it pushes the waiting callback from the queue onto the stack to execute. This prevents network calls from blocking user clicks."

4. Destructuring & Spread Operator (...)

React Daily Use

```
// Object Destructuring: extract properties directly
const student = { name: 'Sautrik', dept: 'CSE', reg: 'RA2511003' };
const { name, dept } = student; // name = 'Sautrik', dept = 'CSE'

// Spread Operator (...) with Arrays: create copy without mutating original
const oldLoans = ['BK1', 'BK2'];
const newLoans = ['BK3', ...oldLoans]; // ['BK3', 'BK1', 'BK2']

// Spread Operator (...) with Objects: update a property immutably
const book = { id: 'BK1', title: 'Clean Code', copies: 4 };
const updatedBook = { ...book, copies: book.copies - 1 }; // copies: 3
```

Module 2: React 18 Core Fundamentals & Hooks

They will ask how React works internally, how state updates trigger renders, and what each Hook does.

1. JSX, Props vs State & One-Way Data Flow

Core React

WHAT IS JSX? JavaScript XML. It lets you write HTML-like syntax inside JavaScript files. Browsers cannot read JSX directly; Vite compiles JSX into standard `React.createElement()` calls.

Concept	Props (Properties)	State
Definition	Data passed down from parent to child.	Data created and managed inside the component.
Mutability	Read-only (immutable)! Child cannot modify props.	Mutable via setter (<code>setCount</code>).
Trigger	Parent passes new values.	Calling <code>setState</code> triggers re-render.

2. `useState` & `useEffect` (The Two Most Important Hooks)

Must Master

`useState` : Stores component memory that survives re-renders.

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(prev => prev + 1)}>Count: {count}</button>;
}
```

`useEffect` : Runs side-effects (API calls, timers, event listeners, camera init).

The **Dependency Array** governs when it runs:

Dependency Array	When does the effect execute?	Real Example in LibraX
<code>useEffect(() => {}, [])</code>	Runs ONCE when component mounts (loads).	Starting the camera scanner when Scanner page opens.
<code>useEffect(() => {}, [book])</code>	Runs on mount + whenever <code>book</code> changes.	Generating a new QR code when user clicks a book.
<i>No array at all</i>	Runs on EVERY single render (Avoid!).	Rarely used; can cause infinite re-render loops!

CRUCIAL INTERVIEW QUESTION: "WHAT IS THE CLEANUP FUNCTION IN USEEFFECT?"

"If an effect starts a subscription, timer, or hardware stream (like our webcam via `html5-qr-code`), we return a cleanup function: `return () => { scanner.stop(); }` . React runs this before the component unmounts to prevent memory leaks and keep hardware from locking up."

3. useRef, useCallback & useContext

Advanced Hooks

useRef : Holds a mutable reference that does NOT trigger a re-render when changed, or grabs a direct DOM node reference.

```
const scannerRef = useRef(null); // Holds the active camera instance
const inputRef = useRef(null); // <input ref={inputRef} /> → inputRef.current.focus()
```

useCallback : Memoizes a function definition so it doesn't get recreated on every render. Useful when passing functions to child components to prevent unnecessary re-rendering.

```
const handleReturn = useCallback((book) => {
  returnBook(book);
}, [borrowedBooks]); // Only recreates function if borrowedBooks changes!
```

useContext : Consumes global data without passing props through 10 levels of children ("prop drilling").

```
const { books, borrowBook } = useLibrary(); // Direct access from any component!
```

4. Virtual DOM & The "key" Prop in Lists

Interview Classic

HOW THE VIRTUAL DOM WORKS: React maintains a virtual copy of the DOM in memory. When state updates, React builds a new virtual tree, runs a **diffing algorithm** against the old tree, and updates *only the changed real DOM nodes* (e.g. changing text from "4 copies" to "3 copies").

INTERVIEW QUESTION: "WHY MUST WE PROVIDE A UNIQUE 'KEY' WHEN MAPPING ARRAYS?"

"React uses the `key` (like `key={book.id}`) to track which specific item was added, moved, or deleted. If you use array indices (`key={index}`), reordering or deleting items causes React to confuse elements, causing subtle UI rendering bugs and poor performance."

⚡ Module 3: Vite vs Create-React-App & Modern Tooling

How Vite Actually Works Under The Hood

Tooling

Feature	Old Webpack / Create-React-App	Modern Vite
Dev Server Start	Slow (30-60s) because it must bundle everything first.	Instant (<300ms) because it serves files over native ES Modules.
Pre-bundling	Slow JavaScript compiler (Babel/Webpack).	esbuild written in Go (10x to 100x faster than JS bundlers).
Production Build	Webpack production minification.	Rollup for clean, highly optimized tree-shaking.
Hot Module Replacement (HMR)	Slows down as project grows larger.	Stays instant regardless of how many files you add.

INTERVIEW QUESTION: "HOW DOES THE BROWSER RUN VITE'S ES MODULES?"

"In our `index.html`, we have `<script type='module' src='/src/main.jsx'>`. Modern browsers natively understand `import` and `export` statements, so Vite only transforms and serves the specific file requested by the browser on demand."

Module 4 & 5: Node.js & Express.js REST APIs

1. What is Node.js? (Single-Threaded, Non-Blocking)

Runtime

WHAT IS IT? Node.js is a C++ program that embeds Google Chrome's **V8 JavaScript Engine** and the **libuv** library, allowing JavaScript to run on servers and access files, databases, and networks.

WHY IS IT FAST FOR I/O? Node is single-threaded for code execution, but it uses **non-blocking asynchronous I/O**. When a database query or network request is pending, Node offloads it to the operating system or worker threads in libuv, continuing to serve other student requests without waiting!



INTERVIEW QUESTION: "WHAT IS COMMONJS VS ES MODULES IN NODE?"

"CommonJS uses `const express = require('express')` and `module.exports = app`. ES Modules use modern `import express from 'express'` and `export default app`. In Node, adding `"type": "module"` in `package.json` enables native ES Modules."

2. Express.js REST API & Middleware Architecture

[Backend Code](#)

WHAT IS MIDDLEWARE? A function that runs between receiving a request and sending a response: `(req, res, next) => { ... next(); }`. It can inspect headers, parse JSON bodies, check authentication, or log requests.

A Complete Express Server Example (Notice `req.params`, `req.body`, `res.json`):

```
const express = require('express');
const cors = require('cors');
const app = express();

// 1. Middleware Chain
app.use(cors()); // Allows frontend on port 5173 to talk to backend
app.use(express.json()); // Parses incoming JSON request body

// 2. GET Route - Retrieve all books (with Query String: /api/books?category=CS)
app.get('/api/books', (req, res) => {
  const category = req.query.category; // Query parameter
  res.status(200).json({ success: true, books: [] });
});

// 3. POST Route - Issue a Book (Accessing JSON Body)
app.post('/api/transactions/issue', (req, res) => {
  const { bookId, studentReg } = req.body; // Request Body

  if (!bookId || !studentReg) {
    return res.status(400).json({ error: 'bookId and studentReg are required' });
  }

  // Return 201 Created
  res.status(201).json({ message: 'Book issued successfully', dueDate: '14 days' });
});

// 4. GET by ID - Route Parameter (e.g., /api/books/BK002)
app.get('/api/books/:id', (req, res) => {
  const bookId = req.params.id; // Route parameter
  res.json({ id: bookId, title: 'Clean Code' });
});

app.listen(5000, () => console.log('Server running on port 5000'));
```

Status Code	Name	Meaning & When to use
200 OK	Success	Standard successful GET, PUT, or DELETE request.
201 Created	Resource Created	Successful POST request that created a new loan or book.
400 Bad Request	Client Input Error	Client sent missing fields, invalid JSON, or exceeded quota.
404 Not Found	Not Found	Requested Book ID or Student record does not exist.
500 Server Error	Internal Error	Uncaught backend code bug or database crash.

Module 6: SQL & Database Query Mastery

If the interviewer says: "Write a SQL query to...", these are the exact queries you need to write on the whiteboard!

1. Schema Creation & Data Insertion (DDL & DML)

SQL Core

Query: Create the `books` table with Primary Key and Constraints:

```
CREATE TABLE books (  
  id VARCHAR(20) PRIMARY KEY,  
  title VARCHAR(255) NOT NULL,  
  author VARCHAR(255) NOT NULL,  
  isbn VARCHAR(50) UNIQUE,  
  category VARCHAR(100) DEFAULT 'General',  
  total_copies INTEGER DEFAULT 5,  
  available_copies INTEGER CHECK(available_copies ≥ 0)  
);
```

Query: Insert a new book record:

```
INSERT INTO books (id, title, author, isbn, category, total_copies, available_copies)  
VALUES ('BK002', 'Clean Code', 'Robert C. Martin', '978-0132350884', 'Computer Science', 5, 3);
```

2. SELECT Queries: Filtering, Pattern Matching & Pagination

Most Frequent

Query: Find available Computer Science books with title search:

```
SELECT id, title, author, available_copies  
FROM books  
WHERE category = 'Computer Science'  
  AND available_copies > 0  
  AND title LIKE '%Code%'      -- % matches any characters before/after  
ORDER BY title ASC             -- Alphabetical order  
LIMIT 10 OFFSET 0;            -- Page 1 (first 10 books)
```

3. UPDATE & DELETE (Modifying Circulation Records)

Inventory

Query: Decrement copy count when issuing a book:

```
UPDATE books  
SET available_copies = available_copies - 1  
WHERE id = 'BK002' AND available_copies > 0;
```

Query: Remove an old returned loan record:

```
DELETE FROM borrowings  
WHERE id = 'loan_999' AND status = 'Returned';
```

4. SQL JOINS: Combining Multiple Tables

Top Interview Topic

WHAT IS A JOIN? Relational databases normalize data into separate tables (e.g. `students`, `books`, `borrowings`) to avoid duplication. A JOIN connects rows from two tables using a common key (Foreign Key).

Query: Get all active loans showing Student Name, Book Title, and Due Date (INNER JOIN):

```
SELECT
  b.id AS loan_id,
  s.name AS student_name,
  s.reg_no,
  bk.title AS book_title,
  b.due_date
FROM borrowings b
INNER JOIN students s ON b.student_id = s.id
INNER JOIN books bk ON b.book_id = bk.id
WHERE b.status = 'Active';
```

The 4 Types of SQL Joins:

Join Type	What it returns
INNER JOIN	Only rows where there is a match in BOTH tables.
LEFT JOIN	ALL rows from the left table, and matching rows from the right table (unmatched become NULL).
RIGHT JOIN	ALL rows from the right table, and matching from left.
FULL JOIN	ALL rows when there is a match in EITHER table.

5. Aggregations (COUNT, GROUP BY, HAVING)

Analytics Queries

Query: Count total books and average available copies per category:

```
SELECT
  category,
  COUNT(*) AS total_titles,
  SUM(total_copies) AS total_volumes,
  AVG(available_copies) AS avg_available
FROM books
GROUP BY category
HAVING COUNT(*) ≥ 2; -- Only categories with 2 or more book titles
```



INTERVIEW QUESTION: "WHAT IS THE DIFFERENCE BETWEEN WHERE AND HAVING IN SQL?"

"WHERE filters individual rows *before* any grouping or aggregation takes place. HAVING filters aggregate groups *after* the GROUP BY clause has run (e.g. `HAVING COUNT(*) > 5`)."

6. SQL Injection & How Prepared Statements Stop It

Security Must-Know

✗ BAD VULNERABLE CODE (String Concatenation):

```
// If attacker enters: " ' OR '1'='1 "  
const query = "SELECT * FROM users WHERE username = '" + userInput + "'";  
// Evaluates to: SELECT * FROM users WHERE username = ' ' OR '1'='1' → Returns ALL users!
```

✓ SECURE CODE (Prepared Statement with Parameter Binding):

```
// In Node.js with better-sqlite3:  
const stmt = db.prepare('SELECT * FROM users WHERE username = ?');  
const user = stmt.get(userInput); // userInput is treated strictly as a string literal, never SQL!
```

Final Quick Revision Cheat Sheet

Subject	Top Interview Topic	What to remember / write in 5 seconds
JavaScript	<code>var</code> vs <code>let</code> vs <code>const</code>	<code>const</code> by default, <code>let</code> if reassigned. Both are block-scoped <code>{}</code> . Avoid <code>var</code> .
JavaScript	<code>=</code> vs <code>===</code>	<code>===</code> checks both value and type without coercion. Always use <code>===</code> .
JavaScript	Array Methods	<code>.map()</code> transforms, <code>.filter()</code> keeps matches, <code>.find()</code> returns first match.
React	Virtual DOM	Lightweight DOM in memory. React diffs it with previous state and updates only changed nodes.
React	<code>useEffect</code> cleanup	Return a function <code>return () => { cleanup(); }</code> to stop timers/cameras and prevent memory leaks.
React	Why <code>key</code> prop?	Gives items stable identities so React diffing knows which element changed, moved, or deleted.
Vite	Why faster than CRA?	Uses browser native ES Modules and Go-based <code>esbuild</code> instead of slow Webpack bundles.
Node.js	Event Loop & I/O	Single-threaded execution with non-blocking async I/O via libuv and background thread pool.
Express	Middleware	Functions with <code>(req, res, next)</code> that inspect or alter request/response before route handler.
SQL	INNER vs LEFT JOIN	<code>INNER JOIN</code> = only matches in both. <code>LEFT JOIN</code> = all from left + matching from right.
SQL	WHERE vs HAVING	<code>WHERE</code> filters rows <i>before</i> grouping; <code>HAVING</code> filters aggregate groups <i>after</i> <code>GROUP BY</code> .
SQL	SQL Injection Fix	Use Prepared Statements with parameter placeholders (<code>?</code>) instead of string concatenation.